

SFA — Crop Book (ספר גידולים): Feature Catalog

Document type: Product information — deep feature catalog **Date:** 2026-06-03 **Status:** Live product (knowledge base + enrichment layer live; calculator UI in active development as S003-P004) **Audience:** NotebookLM research corpus; PM planning reference; feature one-pager

Abstract

The Crop Book (ספר גידולים) is SFA's flagship product: a multi-source agronomic knowledge base for ~66 crops and ~368 varieties that doubles as a **calculator-driven planning tool**. Rather than a static reference, the Crop Book wires its enriched data directly into 14 calculators — covering everything from seed quantity to crop profit comparison — that turn agronomic data into field-ready outputs. Every numeric value carries a source, confidence score, and field state (VALIDATED / UNVALIDATED / MISSING). User-adjustable AssumptionFields (e.g., germination rate 90%, bed width 80 cm) are first-class UI components with inline explainers and linked content. The UX serves two audiences — a gardener Card view and a farmer Table view — with three depth levels per crop (Simple / Full / Drill-down). This document catalogs every current feature and its implementation details.

1. Agronomic knowledge model

1.1 Corpus

Dimension	Value
Crops	~66 baseline crops
Varieties	~368 varieties
Botanical families	Multiple (organized in <code>crop_families</code> table with Hebrew and scientific names)
Primary data language	Hebrew names (<code>name_he</code>), with English (<code>name_en</code>) and Latin (<code>scientific_name</code>)
DB backend	PostgreSQL 15 (waldhomeserver); MySQL read-mirror on uPress for delivery
Schema	6 core tables: <code>crop_families</code> , <code>crops</code> , <code>crop_varieties</code> , <code>crop_variety_source_values</code> , <code>crop_field_enrichment</code> , <code>crop_unit_conversions</code> (plus auxiliary: <code>crop_task_templates</code> , <code>crop_knowledge_notes</code> , <code>crop_attributes</code> , <code>postharvest_storage</code> , <code>planting_calendar</code> , etc.)

1.2 Thirteen-topic taxonomy

Each crop's agronomic profile is organized into 13 content topics:

#	Topic (English)	Hebrew tab label	Content examples
1	Species identity	זהות	Botanical family, scientific name, category (vegetables/herbs/legumes/fruits/cover crops/grains)

#	Topic (English)	Hebrew tab label	Content examples
2	Planting calendar	לוח זמנים	Planting season, days to maturity, harvest window, succession interval
3	Nursery	משתלה	Days in nursery (cell), transplant vs direct seeding, nursery tray type
4	Spacing / population	ריווח	In-row spacing (cm), rows per bed, plants per m ²
5	Yield	יבול	Average yield per bed-meter (kg/m), harvest window min/max days
6	Market / price	כלכלה	Documented price (ILS), price unit (kg / bunch / unit), conversion groups
7	Fertility / nutrition	דישון	Nutrient removal N/P/K/Ca/Mg (kg/ha)
8	Frost / climate	אקלים	Frost tolerance class (very_hardy / hardy / semi_hardy / tender / very_tender / warm)
9	Succession	סוקצסן	Succession interval (weeks), planting season window
10	Harvest	קציר	Harvest window, harvest method notes
11	Postharvest storage	אחסון	Storage method, temperature, humidity, shelf life
12	Equipment / seeder	ציוד	Seeder model, front/rear gear, roller plate
13	Companions / rotation	לוויין	Botanical family rotation hint; companion matrix (future CB-2/3)

1.3 Multi-source enrichment and provenance

Every numeric agronomic field is not a single authoritative value but a **reconciled best estimate** from multiple source tiers:

Source class taxonomy (7 classes, ordered by trust):

Class	Code	Trust weight	Description
Expert override	EX	Hard winner	Direct expert-supplied value; bypasses statistical blend
Nimrod input	NI	Hard winner	File/link supplied by platform owner; treated as EX-tier
Primary reference	PR	0.70	JMF MasterClass Excel data (the primary agronomic baseline)
Operational data	OP	0.55	Real farm data from Tend or field reports
Market-sourced	MK	0.40	Market-derived agronomic observations
Web-research	WR	0.60	Web-grounded AI synthesis (Claude-sourced, labeled UNVALIDATED)
User-contributed	UC	0.15	Community-submitted values, lowest trust

The **reconciler engine** (`organic_market_agent/crop_book/importer/reconciler.py`) applies per-field blend strategies: - `weighted_mean` — weighted average across source classes, with outlier gate (MAD-based, z-threshold 3.5 default) - `hard_winner` — highest-trust class wins outright (used for categorical/interval fields like `succession_interval_weeks`)

The result of reconciliation is stored in `crop_field_enrichment.value_best` with `confidence_score` and `winning_source_class`.

1.4 Provenance cues in the UI

The delivery tier surfaces provenance at three levels:

- **Simple view** — clean value, no cues (the "winning value" only)
- **Full view** — value + field state pill (VALIDATED / UNVALIDATED) + asterisk on low-confidence values
- **Drill-down view** — per-variety source breakdown, confidence score, winning source class, source count

A field with no `value_best` shows "—" (em dash) and a "🕒 בקשו נתון" (request info) prompt — never a misleading zero or a silent gap.

2. Complete / partial crop state

2.1 State machine

Every shown crop is classified as **COMPLETE** or **PARTIAL** based on its mandatory field coverage:

Field state	Condition	UI rendering
VALIDATED	<code>winning_source_class</code> $\in$ {EX, NI} OR <code>confidence_score</code> $\geq \tau$ (with <code>source_count</code> ≥ 1)	Plain value; calculators enabled
UNVALIDATED	Row exists but <code>confidence</code> $< \tau$, or winning class $\in$	Value with asterisk + tooltip "web/low-confidence source"; calculators enabled but flagged
MISSING	No <code>crop_field_enrichment</code> row for the field	"—" + "request info" CTA; calculators needing this field disabled

Confidence threshold $\tau = 0.40$ (configurable). At $\tau = 0.40$: - EX/NI always validate (hard overrides bypass τ) - PR (0.70) and OP (0.55) corroborated values clear the threshold - Lone WB (0.30) or UC (0.15) values do not

Crop-level state: - COMPLETE — every mandatory field is VALIDATED - PARTIAL — at least one mandatory field is UNVALIDATED or MISSING

A PARTIAL crop shows all available data with appropriate asterisks, disabled calculators only where a required book field is MISSING (not merely unvalidated), and "request info" prompts for missing fields. No silent gaps.

2.2 Mandatory fields

The mandatory field set is defined as: every field JMF MasterClass carries, plus every book field a calculator reads. The complete list of 16 mandatory fields:

`days_to_maturity`, `harvest_window_min_days`, `harvest_window_max_days`, `in_row_spacing_cm`, `rows_per_bed`, `avg_yield_per_bed_m`, `documented_price` (+ unit), `planting_season`, `planting_method`, `frost_tolerance_class`, `seeds_per_gram`, `nutrient_removal_n_kg_ha`,

```
nutrient_removal_p_kg_ha , nutrient_removal_k_kg_ha , days_in_nursery_cell ,  
succession_interval_weeks .
```

3. The 14 calculators

All 14 calculators are **pure functions** in `organic_market_agent/crop_book/calculators.py` — no DB access, no I/O, no globals. They accept book values (from `crop_field_enrichment.value_best`), AssumptionFields (with user-adjustable defaults), and direct user inputs, and return typed frozen result dataclasses.

A calculator is **enabled** for a crop if every required book field has a `value_best` (state `VALIDATED` or `UNVALIDATED`). It is **disabled** if any required book field is `MISSING` — and the disabled state shows exactly which field is missing and why. AssumptionFields never disable a calculator (they always have defaults).

3.1 Calculator catalog

Audience codes: G = home gardener · F = small farmer · B = both

#	Calculator name	Audience	Key book fields	AssumptionFields	Key user inputs	Primary output
1	Seed quantity to buy	B	rows_per_bed, in_row_spacing_cm, seeds_per_gram	germination_rate (90%), oversow (1.10x)	bed_length_m, seeds_per_hole	grams of seed to purchase
2	Transplants / seedlings needed	B	rows_per_bed, in_row_spacing_cm	—	bed_length_m	plant count
3	Nursery trays + sow date	F	days_in_nursery_cell	tray_cells (128), oversow (1.10x)	plants (from #2), field_set_date	tray count + tray sow date
4	Sowing date (back-calc from harvest)	B	days_to_maturity, planting_method	—	target_harvest_date	sow date (+ field-set date for transplant crops)
5	Harvest date + window (forward)	B	days_to_maturity, harvest_window_max_days, planting_method	—	sow_date	harvest_start, harvest_end
6	Succession schedule	F	succession_interval_weeks	—	first_sow_date, num_successions or season_end	list of sow dates
7	Beds / bed-meters for target yield	F	avg_yield_per_bed_m	std_bed_length_m (30 m)	target_kg	bed_meters + beds needed
8	Expected yield from an area	B	avg_yield_per_bed_m	—	bed_length_m	yield_kg

#	Calculator name	Audience	Key book fields	AssumptionFields	Key user inputs	Primary output
9	Expected revenue	F	avg_yield_per_bed_m, documented_price, price unit	—	bed_length_m	yield_kg + revenue (ILS)
10	Plant population / spacing layout	B	rows_per_bed, in_row_spacing_cm	bed_width (80 cm)	—	plants_per_m ² + visual grid layout
11	Frost / planting window	B	frost_tolerance_class, days_to_maturity	hardiness_offset (by class)	last_frost_date, first_frost_date	earliest_plant, latest_plant
12	Fertilizer / compost rate	F	nutrient_removal_n_kg_ha (+ P, K informational)	compost_N_pct (1.5%), application_efficiency (0.50)	area_m ²	compost_kg needed + N/ P/K kg
13	Crop profit comparison	F	avg_yield_per_bed_m, documented_price	—	bed_meters (optional)	ranked list of crops by revenue/ margin per meter
14	Seed / input cost	F	grams needed (from #1)	—	seed_price_per_g or (pack_price + g_per_pack)	total seed cost; feeds margin into #13

3.2 Calculator formulas (key examples)

#1 — Seed quantity:

```
plants = round((bed_length_m × 100 / in_row_spacing_cm) × rows_per_bed × seeds_per_hole)
seeds = ceil(plants / germination_rate × oversow)
grams = seeds / seeds_per_gram
```

#7 — Beds for target yield:

```
bed_meters = target_kg / avg_yield_per_bed_m
beds = bed_meters / std_bed_length_m
```

#10 — Plant population:

```
plants_per_m2 = (rows_per_bed / bed_width_m) × (100 / in_row_spacing_cm)
```

#11 — Frost / planting window:

```
offset = hardiness_offset(frost_tolerance_class)
earliest_plant = last_frost - offset [positive offset = plant before last frost]
latest_plant = first_frost - days_to_maturity
```

#12 — Fertilizer rate:

```
area_ha = area_m² / 10_000
N_kg = nutrient_removal_n_kg_ha × area_ha
compost_kg = N_kg / (compost_N_pct × application_efficiency)
```

3.3 Error handling

A calculator raises `CalcUnavailable(field_name)` — a typed exception — if any required book value is `None`. The caller (UI layer) catches this and renders the disabled-calculator state with the missing field name displayed. This is an explicit contract: wrong outputs are impossible because the function refuses to run rather than producing a result from missing data.

3.4 Calculator execution architecture

Default model: **Python computes once at data push time**; results are server-rendered into the delivery-tier payload. Client-side `AssumptionField` overrides `recompute` via a thin JavaScript mirror **only** for the six interactive calculators (#1, #7, #8, #9, #10, #12) that accept live user input changes. The JS mirror is parity-tested against the Python outputs. Date calculators (#3, #4, #5, #6, #11) and #13/#14 are server-render-only in v1.

3.5 Calculator dashboard (/calc/)

A standalone calculator dashboard at `/calc/` provides a shared context strip (crop selector, bed count, target date) that feeds all 14 modules grouped by topic. Every calculator is also embeddable as a modal on individual crop pages. The dashboard supports export stubs (CSV and print).

3.6 Forward API contracts (future modules)

Calculator outputs are typed contracts for downstream modules not yet built:

Calculator outputs	Future consumer
sowing_date, harvest_window, succession schedule (#4, #5, #6)	Planner (CB-2) + Tasks (CB-3) — date anchors
beds_for_target_yield, plant_population (#7, #10)	Planner (CB-2) — bed map
expected_revenue, crop_profit_comparison, seed_input_cost (#9, #13, #14)	Sales/POS (CB-4)
crop_field_enrichment + source provenance	Tend (CB-5) — OP-class write-back

The Crop Book owns agronomic knowledge and calculators only; Planner/Tasks/POS/Tend are downstream consumers of these typed outputs.

4. AssumptionField pattern

4.1 Definition

An **AssumptionField** is a planning assumption — not a per-crop book value — that a calculator needs. Rather than a hard-coded constant or a per-crop DB field, AssumptionFields are **first-class UI components** with:

- A sensible default value (e.g., `germination_rate = 0.90`)
- An inline user override input (recomputes the calculator client-side; never mutates DB)
- A clear, attractive Hebrew explainer text (when/why/how to change it)
- A "read more →" link to a full nimrod.bio blog post

AssumptionFields are defined in `organic_market_agent/crop_book/assumptions.py` in the `ASSUMPTIONS` registry. They never disable a calculator — they always have a default.

4.2 The 8 AssumptionFields

AssumptionField	Default	Used by calculators	Explainer theme	nimrod.bio post required?
<code>germination_rate</code>	90%	#1	Seed viability decay, when to increase oversow, how to test germination	REQUIRED at launch
<code>bed_width</code>	80 cm	#10	JM Fortier 30" → 80 cm standardization rationale (Israeli metric standard)	REQUIRED at launch
<code>oversow</code>	1.10×	#1, #3	Why sow ~10% extra; thinning vs gaps	Optional
<code>tray_cells</code>	128 (by tray type)	#3	Tray sizing choices; cells vs plug size	Optional
<code>std_bed_length_m</code>	30 m	#7	Standard bed length basis for planning	Optional
<code>hardiness_offset</code>	Table by class	#11	Frost tolerance class → safe-plant offset	Optional
<code>compost_N_pct</code>	1.5%	#12	Typical N content of compost; reading a compost analysis	Optional
<code>application_efficiency</code>	0.50	#12	First-year N availability from compost	Optional

4.3 Hardiness offset table (for calc #11)

frost_tolerance_class	Plant before/after last frost
very_hardy, hardy	28 days before last frost
semi_hardy	14 days before last frost
tender	At last frost (0 offset)

frost_tolerance_class	Plant before/after last frost
very_tender, warm	14 days after last frost
null / unknown	At last frost (conservative default)

4.4 Tray cells lookup

Common nursery tray sizes: 50, 72, 104, 128 (default), 200, 242, 288 cells. Tray type is a crop attribute; the AssumptionField defaults to 128 when unknown.

5. Two-audience UX

5.1 Cards view (gardener, גינאי ביתי)

- Default view at /crop-book/
- Crop cards with: name (Hebrew), family, season icons, thumbnail image, key headline values (DTM, spacing, yield)
- Calculators foregrounded: #1 (seed quantity), #2 (transplants), #4 (sow date back-calc), #5 (harvest window forward), #8 (expected yield), #10 (plant population), #11 (frost window)
- Suitable for mobile browsing

5.2 Table view (farmer, חקלאי קטן)

- Compact, multi-row table with sortable columns
- Farmer-specific calculators foregrounded inline per row: #6 (succession schedule), #7 (beds for yield), #9 (expected revenue), #12 (fertilizer rate), #13 (crop profit comparison)
- Higher information density; suited for desktop planning sessions

5.3 Audience switcher

A persistent toggle button ("טבלה" / "כרטיסיות") switches between the two views. State is client-side; same data underlies both views.

6. Depth model: Simple / Full / Drill-down

Each crop page offers three depth levels:

Level	Hebrew label	Content shown
Simple	פשוט	Headline values (DTM, spacing, yield, price) + 3–4 key enabled calculators
Full	מלא	All 13 topic tabs + all enabled calculators + AssumptionField overrides
Drill-down	פירוט	Per-variety detail + source provenance + confidence scores per field + source count

The depth selector persists per session. Drill-down is where data transparency is most visible: every enriched field shows its winning source class (EX/NI/PR/OP/WR/UC), confidence score, and source count.

7. Per-crop content sections

A full crop page (Full depth) includes these sections in order:

1. **Hero + Identity** — Hebrew name, scientific name, botanical family, category, season icons, hero image
2. **Planting calendar** — visual calendar with sow/transplant/harvest windows; season chip filters
3. **Agronomy (crop-level rollup)** — spacing, rows per bed, days to maturity, harvest window, planting method
4. **Harvest & yield** — yield per bed-meter, harvest window detail, harvest notes
5. **Storage** — postharvest temperature, humidity, shelf life, storage method
6. **Companions** — botanical family rotation hint (informational; full companion matrix is future CB-2/3)
7. **Notes** — public knowledge notes sourced from JMF MasterClass NotebookLM extracts (24 JSON cache files, ~37 source sheets)
8. **Varieties** — variety grid with per-variety agronomy deltas (where varieties differ from crop-level baseline)

8. How book values bind into calculators (SFA_CROP_BOOK)

The data flow from source to calculator output:

```
JMF Excel / EX overrides / WR fills / UC contributions
    ↓
crop_variety_source_values (raw source rows)
    ↓
enrichment_runner + reconciler (weighted blend + outlier gate)
    ↓
crop_field_enrichment.value_best (ONE winning value per variety × field)
    ↓
sfa_ingest_push.py (pushes value_best + field_state + ASSUMPTIONS to MySQL read-mirror)
    ↓
delivery tier payload (Slim4/PHP reads MySQL → renders calculators with book values)
    ↓
AssumptionField override (JS recomputes interactive calcs client-side)
    ↓
User sees: calculator result derived from real agronomic data
```

Key principle: the delivery tier **reads ONE winning value** (`value_best`) for every displayed agronomic number. It never re-resolves the source hierarchy — that logic lives in the Python reconciler. The UI is a read-only presentation layer.

9. Export and planning outputs

- **CSV export** — the calculator dashboard exposes a CSV export hook for planning outputs (calculator results across multiple crops)
- **Print** — print-optimized layout available from the dashboard

- **Future typed outputs** — calculator result dataclasses are designed as stable contracts for downstream modules (Planner bed-map, Tasks sow/harvest sequences, Sales/POS revenue tracking)

10. Search

A full-text search at `/search/` covers crop names (Hebrew, English, scientific) and variety names. Results link directly to individual crop pages.

11. Technical implementation notes

Component	Location	Role
<code>organic_market_agent/crop_book/calculators.py</code>	walldhomeserver Python package	14 pure-function calculators
<code>organic_market_agent/crop_book/assumptions.py</code>	walldhomeserver Python package	ASSUMPTIONS registry, tray_cells, hardiness_offset tables
<code>organic_market_agent/crop_book/calculator_meta.py</code>	walldhomeserver Python package	Per-calc audience/required-fields/assumption-keys metadata + <code>calc_enabled()</code>
<code>organic_market_agent/crop_book/field_policy.py</code>	walldhomeserver Python package	FIELD_POLICY dict — blend strategy per field
<code>organic_market_agent/crop_book/importer/reconciler.py</code>	walldhomeserver Python package	Weighted-mean + hard-winner reconciler, outlier gate
<code>organic_market_agent/publisher/sfa_ingest_push.py</code>	walldhomeserver Python package	Pushes value_best + field_state + ASSUMPTIONS to uPress MySQL
<code>sfa_delivery/templates/pages/book_crop.php</code>	uPress Slim4 front-end	Per-crop detail page (8 sections)
<code>sfa_delivery/templates/pages/book_table.php</code>	uPress Slim4 front-end	Farmer table view
<code>sfa_delivery/templates/pages/calc_dash.php</code>	uPress Slim4 front-end	Calculator dashboard
<code>sfa_delivery/templates/pages/book_entry.php</code>	uPress Slim4 front-end	Crop Book index / Cards view

Sources: *CALCULATOR_CATALOG_v1.0.0.md*, *LOD400_spec.md* (SFA-S003-P004-WP-CB-1), *MANDATORY_FIELD_SCHEMA_v1.0.0.md*, *GAP_FILL_PLAN_v1.0.0.md*, *organic_market_agent/crop_book/assumptions.py*, *organic_market_agent/crop_book/calculator_meta.py*, *organic_market_agent/crop_book/views.py*, *sfa_delivery/templates/pages/book_crop.php*, *sfa_delivery/templates/pages/calc_dash.php*, *_aos/roadmap.yaml*.